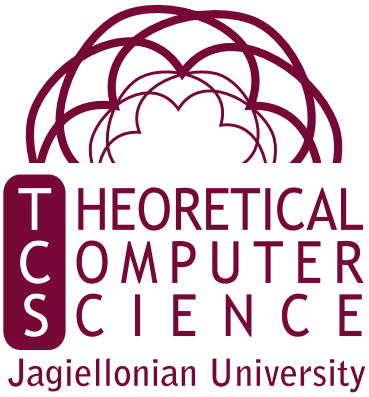
Jagiellonian University in Kraków
Faculty of Mathematics and Computer Science
Theoretical Computer Science Department

Viola-Jones Face Detection Algorithm

Helena Arendacz
helena.arendacz@student.uj.edu.pl

https://github.com/arendaczh/viola-jones-classifier

Advisor:
dr Maciej Ślusarek
Theoretical Computer Science Department



Abstract

Face detection remains a relevant problem in computer vision due to its wide range of applications. This paper explores the Viola-Jones object detection framework, discusses the machine learning and optimization techniques it introduces, along with its underlying mathematical foundations.

1. Introduction

Face detection is a process of identifying whether a given image contains a face and, if it does, returning the location of all faces. The task is regarded nontrivial as images may differ by lightning, size, saturation, etc. To make matters worse, human faces tend to differ as people may have different skin color, glasses, expressions, and poses in pictures. Nevertheless, there are many better or worse method for tackling this problem. In this paper, we discuss **Viola-Jones object detection framework**, a face detection method proposed by Paul Viola and Michael Jones in 2001 [1]. It is notable due to its efficiency and short classification time in comparison to other classification methods [2], and its performance can be successfully enhanced using modern frameworks [3, 4]. Succinctly, the algorithm consists of constructing an integral image and employing it to train a large set of classifiers, that are later combined into a final, effective classifier using a technique called attentional cascade.

The objective of this paper is to implement this framework by hand and to explore the mathematical foundations of it. We begin with a careful examination of techniques proposed by the authors, such as Haar-like features, integral image, AdaBoost, and their revolutionary attentional cascade. Then, we discuss additional implementation details, encountered predicaments, employed performance metrics, and the performance of the implemented model.

2. Viola-Jones classifier

Let us examine the entire construction of a Viola-Jones classifier. In the original paper, the model is trained on 24×24 pixel images and is employed to predict whether they contain a human face.

To determine if an input image of arbitrary size contains a face, the features are rescaled and calculated for plenty of small sub-frames. If the classifier cascade detects a face for any sub-frame, the image is considered to contain a face. Furthermore, this method enables locating all faces and marking their placement in the picture.

2.1. Features

The Viola-Jones uses a Haar-like features - a scalar products between an image I and a pattern P. Descriptively speaking, each pixel taken into account by a given feature (active pixel) is assigned one of two colors - black or white - and the value is computed as

$$\sum_{p \in \{\text{active black pixels}\}} I[p] - \sum_{p \in \{\text{active whire pixels}\}} I[p].$$

All images ought to be mean and variance normalized. In practice, only tree types of features are considered:

- *a two-rectangle feature* - the difference between the sum of the pixels within two adjacent rectangles,
- *a three-rectangle feature* - the difference between the sum of pixels within two rectangles and their intersection,
- *a four-rectangle feature* - the difference between diagonal pairs of rectangles.

Therefore, there are five categories of features, as for two-rectangle and three-rectangle features we must consider both horizontally and vertically adjacent rectangles.

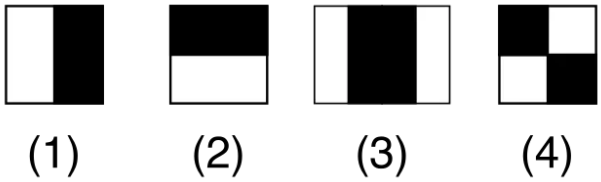


Figure 1: Example Haar-like features

As we want to calculate *all* such features, we need an efficient method to do so.

2.2. Integral Image

Rectangle features can be calculated rapidly using an intermediate, easily precomputed representation called an integral image. For an image I of size $n \times m$ we define an integral image II as:

$$II(x, y) = \sum_{i \leq x,\ j \leq y} I(i, j).$$

It can be computed using dynamic programming in $O(nm)$ as

$$II(x + 1, y + 1) = II(x, y + 1) + II(x + 1, y) - II(x, y).$$

Furthermore, to find a sum of pixels in a (a, b) (c, d) rectangle $R_{a,b,c,d}$ it suffices to calculate

$$I(c, d) - I(c, b) - I(a, d) + I(a, b).$$

Therefore, given an integral image representation of a picture, Haar-like features can be effectively calculated in constant time $O(1)$.

Finally, we can move to discussing the learning process of the previously mentioned classifiers.

2.3. Boosting

Boosting is a highly intuitive approach to machine learning problems. It stems from an idea that compiling many weak models may result in an accurate model that performs well on a given task. The AdaBoost (short for Adaptive Boosting) algorithm was proposed by Yoav Freund and Robert Schapire in 1995 [5]. Given m training examples $(x_i, y_i) \in \mathscr{X} \times \{-1, 1\}$ for some domain $\mathscr{X}$ the goal is to train a *strong classifier* over the course of T rounds. On each round $t = 1, 2, \ldots, T$, a probability distribution D_i is computed over the m training examples and employed to derive

a *weak classifier* $h_t : \mathscr{X} \to \{-1, 1\}$ via a *weak learning algorithm*. The aim of the weak learning algorithm is to find a weak classifier with low weighted error ϵ_t relative to D_t. The final strong classifier is computed as a sign of weighted combination of weak classifiers

$$F(x) = \sum_{t=1}^{T} \alpha_t h_t(x).$$

Equivalently, $F(x)$ is computed as a weighted majority vote of weak classifiers. The absolute value $|F(x)|$ is called a *margin* and represents the confidence of the classifier in its prediction. [6]

Define an exponential loss function

$$L(y, f(x)) = \exp(-yF(x)).$$

As usual, the objective is to find

$$\min_{h_t,\ \alpha_t} \sum_{i=1}^{m} w_i(t) L\left(y_i, \sum_{t=1}^{T} \alpha_t h_t(x_i)\right),$$

where $w_i(t)$ are the importance weights set for training examples. The AdaBoost iteratively finds parameters (h_t, α_t) based on the previously found and set parameters. Additionally, the learned probability distribution D_i is expressed as

$$D_i(t) = w_i(t) \exp\left(-y_i \sum_{s=1}^{t-1} \alpha_s h_s(x_i)\right).$$

Let Z_t be the weighted exponential loss obtained by a t-member committee. We have

$$Z_{t+1} = \min_{h_t,\ \alpha_t} \sum_{i=1}^{m} w_i(t) \exp\left(-y_i \sum_{s=1}^{t} \alpha_s h_s(x_i)\right) = \min_{h_t,\ \alpha_t} \sum_{i=1}^{m} D_i(t) \exp(-y_i \alpha_t h_t(x_i)).$$

Notice that the larger the loss was on the training example in the previous rounds, the larger is its contribution to the loss minimized in the current round. Effectively, the algorithm focuses on the poorly performing instances. Divide the dataset based on $y_i h_t(x_i)$:

$$\begin{aligned} Z_{t+1} &= \min_{h_t,\ \alpha_t} \sum_{i=1}^{m} D_i(t) e^{-\alpha_i} \mathbf{1}\left[y_i h_t(x_i) = 1\right] + \sum_{i=1}^{m} D_i(t) e^{\alpha_i} \mathbf{1}\left[y_i h_t(x_i) = -1\right] \\ &= \min_{h_t,\ \alpha_t} \ e^{-\alpha_i} \sum_{i=1}^{m} D_i(t) + \left(e^{\alpha_i} - e^{-\alpha_i}\right) \sum_{i=1}^{m} D_i(t) \mathbf{1}\left[y_i h_t(x_i) = -1\right] \\ &= Z_t \min_{h_t,\ \alpha_t} \ e^{-\alpha_i} + \left(e^{\alpha_i} - e^{-\alpha_i}\right) \sum_{i=1}^{m} \frac{D_i(t)}{Z_t} \mathbf{1}\left[y_i h_t(x_i) = -1\right]. \end{aligned}$$

Thus the minimization process can be carried out in two steps:

$$Z_{t+1} = Z_t \min_{\alpha_t} \left(\min_{h_t} \; e^{-\alpha_i} + \left(e^{\alpha_i} - e^{-\alpha_i}\right) \sum_{i=1}^{m} \frac{D_i(t)}{Z_t} \mathbf{1}\left[y_i h_t(x_i) = -1\right] \right).$$

First, we minimize Z_{t+1} with respect to h_t:

$$h_t = \arg\min_h \; \sum_{i=1}^{m} \frac{D_i(t)}{Z_t} \mathbf{1}\left[y_i h(x_i) = -1\right] := \arg\min_h \; \epsilon_t,$$

and then proceed with minimizing with respect to α_t :

$$\alpha_t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}.$$

We finish the process with updating the weights:

$$w_i(t+1) = \frac{D_i(t+1)}{Z_{t+1}}.$$

Algorithm 1 AdaBoost

Initialize the initial weights $w_i(1)$
for $t = 1$ to T **do**
    Train the best decision stump h_t and its weighted error $\epsilon_t = \sum_i w_i(t)\mathbf{1}[h_t(x_i) \neq y_i]$
    if $\epsilon_t = 0$ **then**
        return $\operatorname{sgn}(h_t(\cdot))$
    else
        $\alpha_t \leftarrow \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$
        Update the weights:

$$w_i^{(t+1)} \leftarrow \frac{w_i^{(t)}}{2} \left(\frac{\mathbf{1}[h_t(x_i) \neq y_i]}{\epsilon_t} + \frac{\mathbf{1}[h_t(x_i) = y_i]}{1-\epsilon_t} \right)$$

return $\operatorname{sgn}\left(\sum_t \alpha_t h_t(\cdot)\right)$

2.4. Improved boosting

Let us take a closer look at the boosting algorithm in the special case of face detection. The algorithm constructs a large strong classifier that must take into account a plethora of features to be effective. However, this approach is computationally expensive. In general, every window *might* be positive, but for a given image only a small fraction of windows will contain a face, even in face-heavy examples. Thus, another intuitive strategy can be applied. Instead of one enormous classifier, we can construct a list of smaller, specialized classifiers. The objective of each of them will be to reject a fraction of the windows as not containing a face while at the same time retaining as much windows that contain faces as possible. This approach is called an **attentional cascade** and was introduced in the same paper as the Viola-Jones algorithm [1]. In formal terms, the cascade

consists of N strong classifiers that start small and simple, and gradually become more complex and refined. Each classifier h_n rejects $(1 - \gamma_n)$ fraction of the input windows as negative examples, while maintaining $(1 - \beta_n)$ fraction of the positive windows. In probabilistic terms:

$$p\left(h_n(x) = 0 \mid y = 1\right) \leq \beta_n \ \wedge \ p\left(h_n(x) = 1 \mid y = 0\right) \leq \gamma_n,$$

so both false negative error and false positive errors are upper-bounded. Therefore, as the number of layers grows, the overall classification rate is expected to converge to 1.

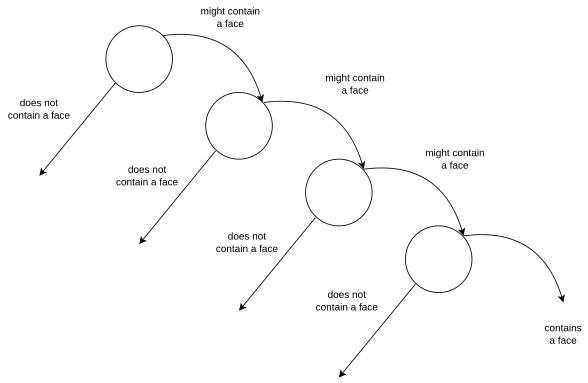


Figure 2: Attentional Cascade

The AdaBoost algorithm by itself does not favor neither the false positive nor the false negative error types. To enable classifiers that are prone to vote in favor or against faced more often than not, we introduce an auxiliary **control** parameter $s \in [-1, 1]$ that shifts the classifier in one direction:

$$F_s(x) = \sum_{t=1}^{T} \alpha_t (h_t(x) + s)$$

3. Dataset

The training set was constructed as a combination of two larger datasets. Images containing faces were downloaded from the **Labeled Faces in the Wild** dataset, which contains over $13,000$ samples. Pictures without faces come from the **CIFAR-10** dataset, which consists of over $50,000$ various images that, among other things, contain animals, vehicles, and flowers. For an effective training process, $3,500$ images were randomly sampled from each set, converted to greyscale, and normalized pixel-wise to have a mean of $\mu = 0$ and a variance of $\sigma^2 = 1$.

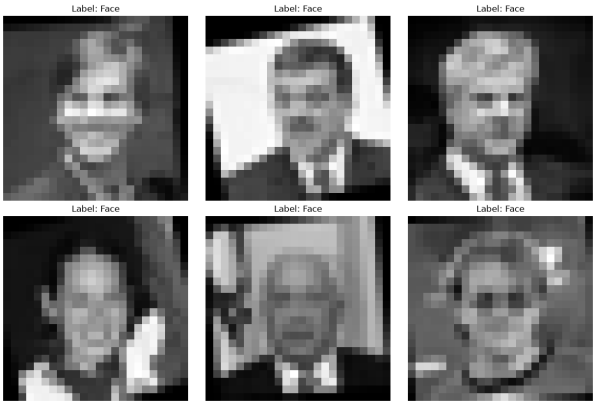


Figure 3: Example faces sampled from the dataset

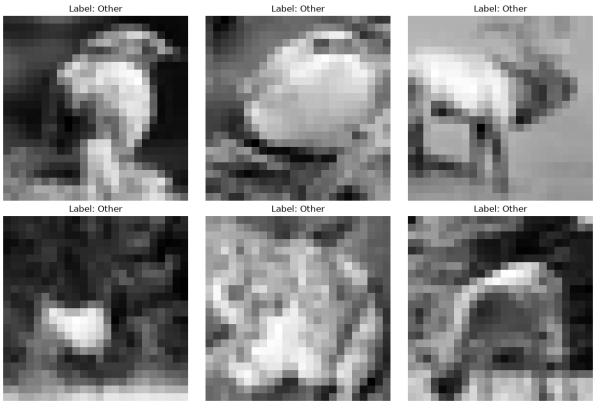


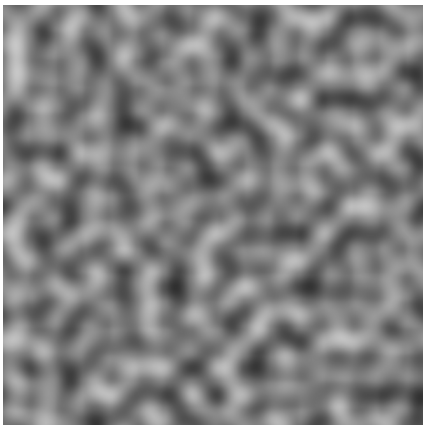
Figure 4: Example non-faces sampled from the dataset

4. Encountered Problems

During the attentional cascade learning process, sub-windows deemed negative are discarded from the training pool. To balance the number of negative samples in the training set, new auxiliary data is introduced. At the introductory stage of the training, samples were generated according to a normal distribution $\mathcal{N}(128, 50)$, clipped to the range $[0, 255]$, and normalized:

$$x \leftarrow \frac{x - \mu}{\sigma^2}.$$

This approach was proven to be ineffective - at approximately 5-th stage, the cascade tended to overfit, and an overwhelming majority of generated samples were rejected at the current stage of the cascade, and as a consequence, were not passed to the succeeding stages. To mitigate this, new negative sample generation techniques were introduced. The first idea was to use more refined random image generation techniques. Ultimately, generation using **Perlin noise** was chosen [7, 8]. Perlin noise is a type of gradient noise, primarily used to generate terrain and assist in the creation of image textures. In short, it serves to create more complex, realistic, but still random images. Finally, extending the training set was considered. More non-face samples were downloaded, Haar-like features were calculated, and a random subset of the features was selected.



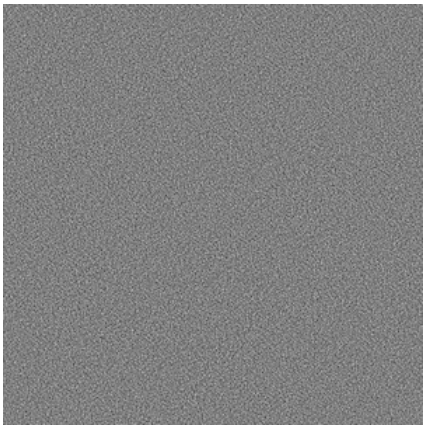


Figure 5: Pictures presents example data generated with the usage of Perlin noise. Sources: 1, 2

.

5. Results

The cascade has been trained with various hyperparameter settings - most notably, with different maximal number of stages. As seen in figures 6, 7, 8, 9, during the whole training process both training and test false positive error gradually decrease. Furthermore, the overall curve shapes suggest that continuing the training process would result in further drop. However, it comes with a cost - the true positive rate starts the training process as 1.0 and decreases.

The cascade starts with almost negligible false negative error rate and high false positive rate. As it begins with classifying all images positively, the false positive error decreases gradually, sacrificing the false negative error.

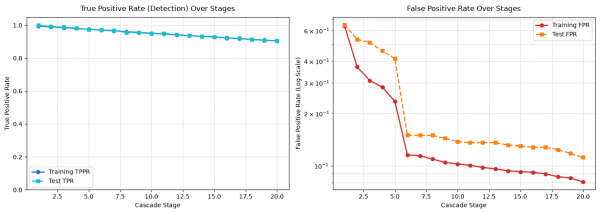


Figure 6: `viola_jones_cascade_14544268`

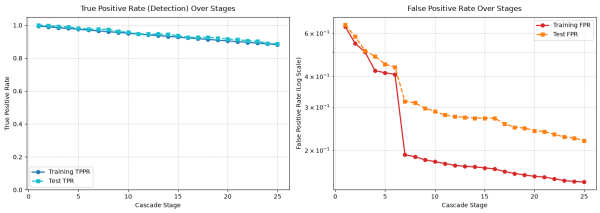


Figure 7: `viola_jones_cascade_7514148`

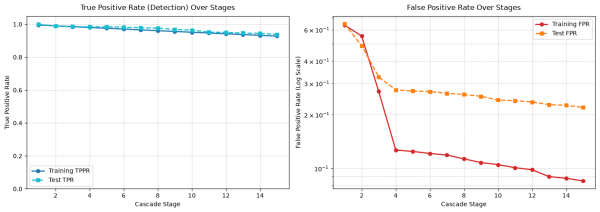


Figure 8: `viola_jones_cascade_3299474`

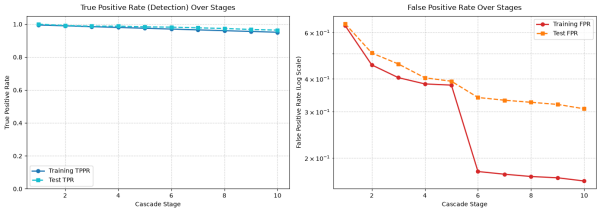


Figure 9: `viola_jones_cascade_5920259`

As a consequence, the overall error rate plummets during the initial stages, but later stagnates and even increases a little 5. Its primary cause is balancing preserving high TPR and decreasing

FPR, and, as FPR drops incomparably faster than TPR, such tradeoff is acceptable and even desirable. It is also worth mentioning that the decrease pace differs visibly between trainings. As a possible culprit we may propose AdaBoost dependency on the weight initialization:

$$w_i(1) = \frac{1}{2m}, \frac{1}{2l}$$

for $y_i = 0, 1$ respectively, where m and l are the number of negatives and positives respectively [1].

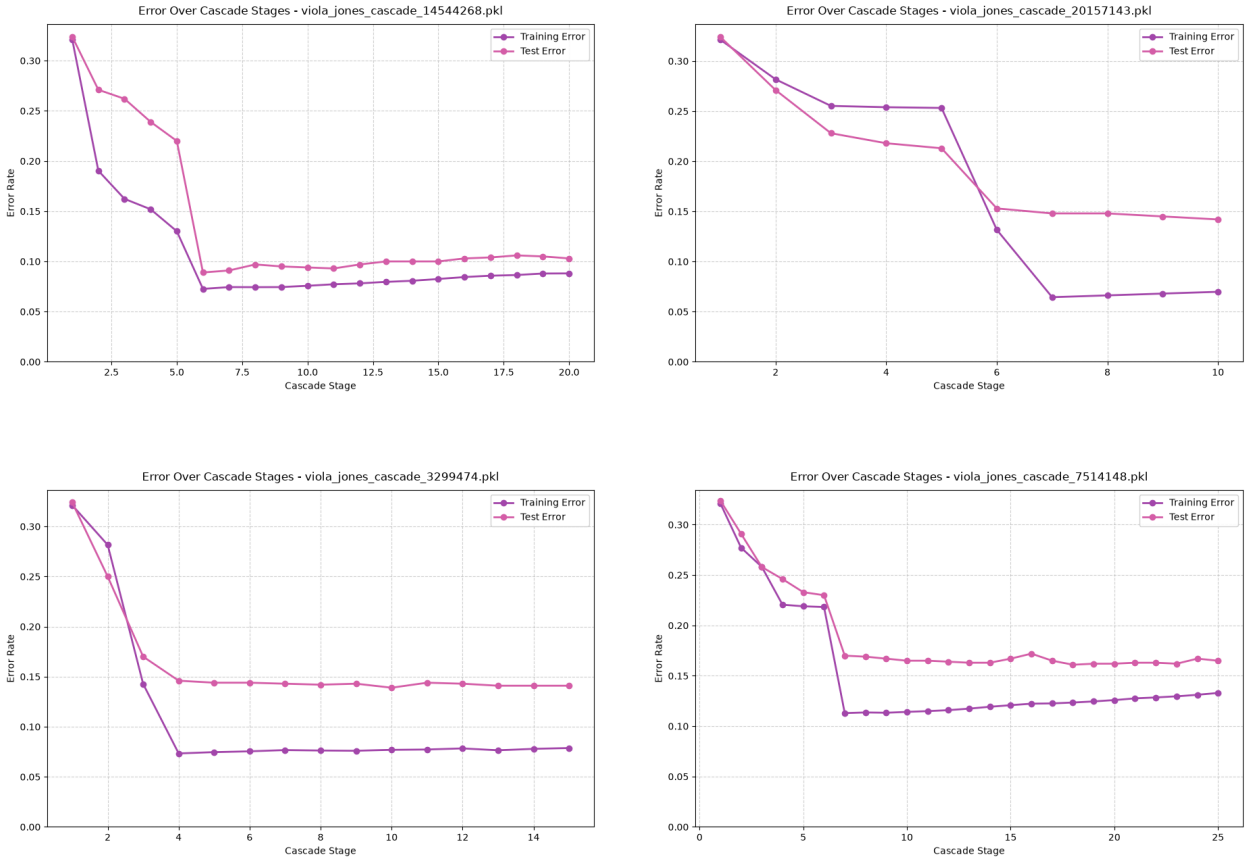


Figure 10: Error Over Cascade Stages

5.1. Rejection rate

The primary advantage of the Viola-Jones classifier is its speed-to-accuracy ratio. Thus, it seems natural to measure the expected number of weak classifiers evaluated per negative sub-window:

$$\mathbb{E}\left[\#\{\text{weak classifiers per window}\}\right] = n_1 + \sum_{t=2}^{T} n_t \prod_{j=1}^{t-1} \text{FPR}_j,$$

where n_t is the number of weak classifiers ensembled to create the t-th strong classifier, and FPR_j - the false positive rate at the j-th stage. Effectively, $\prod_{j=1}^{t-1} \text{FPR}_j$ computes the fraction of negative sub-windows that are falsely rated *positive* and persist in the dataset through first $t-1$ stages.

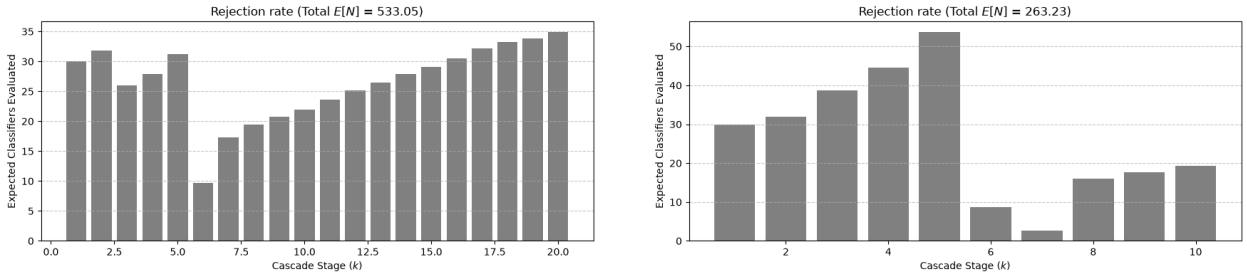


Figure 11: Rejection rate

Load distribution differs significantly between trained classifiers. When a classifier is small, first few stages reject most of the negative sub-windows. With larger classifiers a similar phenomenon takes place, but further layers of the cascade tend to specialize, reject more complex sub-windows, and therefore grow in size trying to reach more and more strict false positive rate objectives.

5.2. Margin

AdaBoost learns to maximize margin

$$y_i H(x_i)$$

and thus confidence in its prediction. As $y_i \in \{-1, 1\}$, the algorithm focuses on increasing $|H(x)|$ for each of the training examples x.

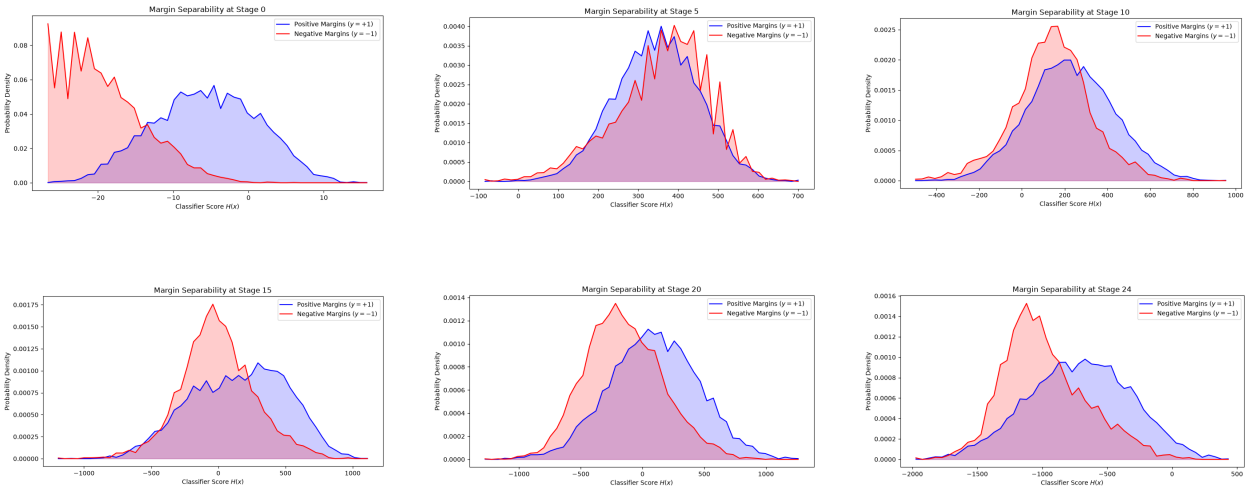


Figure 12: Margin change during the training process

Let us observe how the margin changes along the subsequent stages 12. At the beginning of the training, the cascade struggles to distinguish positive from negative samples. At stages 5 or 10 the distributions of margins for positive and negative samples are almost identical, with similar variations and almost identical means. However, at the latter stages the distributions of these margins emerge as two separate normal-like curves with distinct means and variances. It suggests that the learning process brings desirable effects and the cascade finally distinguishes negative from positive sub-windows.

5.3. ROC curve

ROC (receiver operating characteristic) curve is a graph of true positive rates versus false positive rates for different threshold in binary classification. It suggests model's ability to distinguish between positive and negative classes. **AUC** (area under the ROC curve) is a numerical representation of it - the closer the AUC is to 1.0, the better.

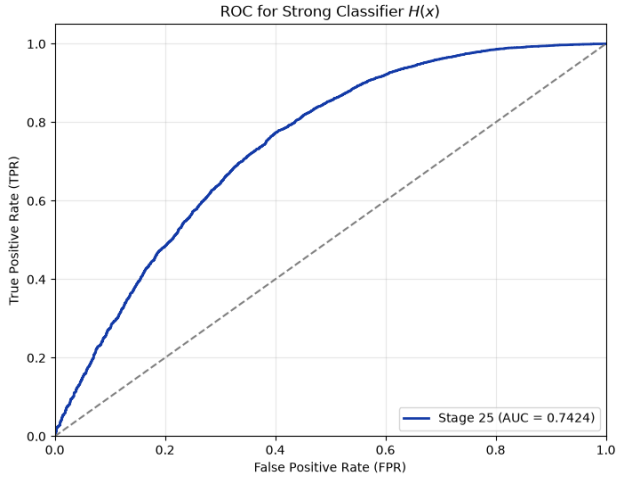


Figure 13: ROC curve

The curve shape and AUC value suggests that model gradually learns to distinguish positive samples from negative values. However, there is still some room for improvement.

6. Conclusion

We have explored and discussed a revolutionary framework for face detection tasks. At first glance simple and straightforward approach of boosting was extended to a more advanced attentional cascade. In the original paper, authors built a huge classifier that exceeded contemporary benchmarks. However, due to hardware limitations, it is difficult to construct a comparably huge and effective classifier. Nevertheless, the trained model learns to effectively separate images containing faces from these that do not. It seems to detect structures typical for faces and, what's important, gives hope that with a larger dataset and better hardware its performance will significantly improve.

References

[1] P. Viola, M. Jones, Rapid object detection using a boosted cascade of simple features, in: Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR), IEEE, 2001.

[2] S. K. Mondal, I. Mukhopadhyay, S. Dutta, Review and comparison of face detection techniques, in: Proceedings of International Ethical Hacking Conference 2019, Advances in Intelligent Systems and Computing, Springer Singapore, 2020. doi:10.1007/978-981-15-0361-0_1.

[3] M. N. Chaudhari, M. Deshmukh, G. Ramrakhiani, R. Parvatikar, Face detection using viola jones algorithm and neural networks, in: 2018 Fourth International Conference on Computing Communication Control and Automation (ICCUBEA), IEEE, 2018. doi:10.1109/ICCUBEA.2018.8697768.

[4] M. Da'san, A. Alqudah, O. Debeir, Face detection using viola and jones method and neural networks, in: 2015 International Conference on Information and Communication Technology Research (ICTRC), IEEE, 2015. doi:10.1109/ICTRC.2015.7156416.

[5] Y. Freund, R. E. Schapire, A decision-theoretic generalization of on-line learning and an application to boosting, in: Computational Learning Theory (EuroCOLT '95), Springer, 1995, pp. 23–37. doi:10.1007/3-540-59119-2_166.

[6] R. E. Schapire, Explaining AdaBoost, in: Empirical Inference: Festschrift in Honor of Vladimir N. Vapnik, Springer Berlin Heidelberg, 2013, pp. 37–52. doi:10.1007/978-3-642-41136-6_5.

[7] K. Perlin, An image synthesizer, in: Computer Graphics (SIGGRAPH '85 Proceedings), Vol. 19, 1985, pp. 287–296.

[8] P. Vigier, Perlin noise with numpy.
URL https://pvigier.github.io/2018/06/13/perlin-noise-numpy.html